T3000 API & Protocol

Message 11: DELETE_IMAGE · built Nov 13, 2026

Message 11: DELETE_IMAGE

The DELETE_IMAGE message removes previously uploaded graphic image files from the server.

When to Use:

- Cleaning up unused images
- Removing old graphics assets
- Managing storage space

Overview

Action: `DELETE_IMAGE` (11) **Direction:** Frontend → Backend **Location:** BacnetWebView.cpp line 3012
Purpose: Delete image files from uploads folder

Request Format

JSON Structure:

```JSON
{
  "action": "DELETE_IMAGE",
  "data": "/uploads/floor-plan.png"
}
```

Request Parameters

Parameter	Type	Required	Description
`data`	string	Yes	Relative path to image file

Path Format:

- Must start with `/uploads/`
- Example: `/uploads/image.png`
- Full server path: `ResourceFile\webview\www\uploads\image.png`

Response Format

No response - Silent deletion

Implementation

Location: BacnetWebView.cpp line 3012

C++

```
case DELETE_IMAGE:
{
    const std::string file_path = Json::writeString(builder, json["data"]);
    CString web_image_folder = g_strExePth + _T("ResourceFile\\webview\\www");
    CString temp_delete_file_path;
    CString temp_file_name = file_path.c_str();

    // Trim quotes from JSON string
    temp_file_name = temp_file_name.TrimLeft(_T("\""));
    temp_file_name = temp_file_name.TrimRight(_T("\""));

    // Build full path
    temp_delete_file_path = web_image_folder + temp_file_name;

    // Check if file exists and delete
    CFileFind temp_find;
    if (temp_find.FindFile(temp_delete_file_path))
    {
        DeleteFile(temp_delete_file_path);
    }
}
```

Frontend Usage

Delete Single Image

TYPESCRIPT

```typescript
const deleteImage = (imagePath: string) => {
  const message = {
    action: 'DELETE_IMAGE',
    data: imagePath  // e.g., "/uploads/image.png"
  };

  sendMessage(JSON.stringify(message));
};

// Usage
deleteImage('/uploads/old-image.png');
```

Delete with Confirmation

TYPESCRIPT

```typescript
const deleteImageWithConfirm = async (imagePath: string, imageName: string) => {
  const confirmed = confirm(`Delete image "${imageName}"?`);

  if (confirmed) {
    deleteImage(imagePath);

    // Remove from UI immediately (optimistic update)
    removeImageFromList(imagePath);
  }
};
```

Batch Delete

TYPESCRIPT

```typescript
const deleteMultipleImages = (imagePaths: string[]) => {
  imagePaths.forEach(path => {
    deleteImage(path);
  });
};
```

Validation

```typescript
const validateImagePath = (path: string): boolean => {
  // Must start with /uploads/
  if (!path.startsWith('/uploads/')) {
    console.error('Invalid path: must start with /uploads/');
    return false;
  }

  // Check for path traversal attempts
  if (path.includes('..')) {
    console.error('Invalid path: path traversal not allowed');
    return false;
  }

  return true;
};
```

Safety Considerations

No Confirmation

The backend deletes files immediately without confirmation. Implement confirmation dialogs in the frontend.

No Undo

Deleted files cannot be recovered. Consider implementing:

- Confirmation dialogs
- Trash/recycle bin functionality
- Usage tracking (don't delete if used in graphics)

Silent Failure

If file doesn't exist, deletion fails silently. No error is returned.

Use Cases

Graphics Manager

TYPESCRIPT

```typescript
const GraphicsManager: React.FC = () => {
  const [images, setImages] = useState<string[]>([]);

  const handleDelete = (imagePath: string) => {
    if (confirm('Delete this image?')) {
      deleteImage(imagePath);
      setImages(images.filter(img => img !== imagePath));
    }
  };

  return (
    <div>
      {images.map(img => (
        <div key={img}>
          <img src={img} alt="" />
          <button onClick={() => handleDelete(img)}>Delete</button>
        </div>
      ))}
    </div>
  );
};
```

See Also

- SAVE_IMAGE - Uploading images
- SAVE_GRAPHIC_DATA - Using images in graphics